Complete Study Guide:

Deep Q-Learning for Continuous State Space Control

A visual, interview-ready report based on the supplied study guide

Core idea	Use neural networks to approximate Q-values when the state space is continuous and too large for a lookup table.
Main refinements	Efficient state-only architecture, epsilon-greedy exploration, mini-batches, and soft updates.
Practical lens	The notes emphasize intuition, stability, hyperparameter sensitivity, and the sim-to-real gap.

Prepared as a study and revision report

Learning Map

Color	Major section	What it answers
Coral	1. Tutor technique	How the lecture builds intuition and complexity
Teal	2. Continuous state spaces	Why state vectors replace discrete tables
Amber	3. Lunar Lander	How actions, rewards, and state are encoded
Blue	4. Deep Q-Learning	How Bellman targets turn RL into supervised learning
Purple	5. DQN refinements	How architecture and exploration improve learning
Orange	6. Stability refinements	Why mini-batches and soft updates help
Green	7. Practical perspective + interview prep	What matters in real systems and interviews

The lecture rhythm at a glance

- Physical anchors first: the toy truck and helicopter make high-dimensional state vectors intuitive.
- Complexity is scaffolded from 1D discrete states to continuous, multi-dimensional vehicle control.
- DQN is framed as supervised learning through Bellman-generated targets.
- Questions are anticipated and answered immediately to close logical gaps.
- The practical message is deliberately pragmatic: RL works, but it is finicky and harder to transfer to real robots.

1. Continuous State Spaces

Intuition & motivation

Many real-world robotic control problems cannot be represented by a tiny set of discrete positions. A simplified Mars Rover can occupy one of six discrete locations, but a real rover moves along a continuous trajectory, where values such as 2.7 km or 4.8 km are valid.

Dimensionality grows with physical realism



Increasing physical realism and state dimensionality

Figure 1. The guide progresses from a 1D discrete toy model to 12-dimensional flight control. The progression is the conceptual bridge from tabular methods to function approximation.

Toy truck: 6-number state vector

- x-position
- y-position
- Orientation angle (theta)
- Velocity in x (x dot)

- Velocity in y (y dot)
- Angular velocity (theta dot)

Toy helicopter: 12-number state vector

- North-South position (x)
- East-West position (y)
- Height above ground (z)
- Roll (phi)
- Pitch (theta)
- Yaw (omega)
- Linear velocity in x (x dot)
- Linear velocity in y (y dot)
- Linear velocity in z (z dot)
- Angular velocity of roll (phi dot)
- Angular velocity of pitch (theta dot)
- Angular velocity of yaw (omega dot)

Key takeaway: A continuous-state MDP uses a multi-dimensional vector of continuous numbers as the input to a policy, which maps state information to an action.

2. The Lunar Lander Environment

State and action space

The Lunar Lander is a simulated rocket-landing task. On each time step, the agent chooses among four discrete actions while observing an 8-dimensional state.

Action	Effect
nothing	Gravity and inertia pull the lander down
left	Fire the left thruster; push the lander right
main	Fire the main bottom engine; push the lander upward
right	Fire the right thruster; push the lander left

The 8-dimensional state

Type	Variables
Continuous	x, y, x dot, y dot, theta, theta dot
Binary	left leg grounded (l in {0,1}), right leg grounded (r in {0,1})

Reward design

Event	Reward signal
Successful/soft landing	+100 to +140 based on flight performance and centering
Moving closer to pad	Positive
Drifting away	Negative
Main thruster	-0.3
Left/right thruster	-0.03

Event	Reward signal
Each grounded leg	+10
Crash	-100

Optimization goal: learn a policy pi(S) that maps state to action and maximizes the discounted sum of rewards, with gamma = 0.985 in the guide.

3. Deep Q-Learning (DQN)

Why a neural network?

A Q-table becomes impractical in a continuous state space. The guide instead treats the Q-function as a function approximated by a neural network, Q(s,a).

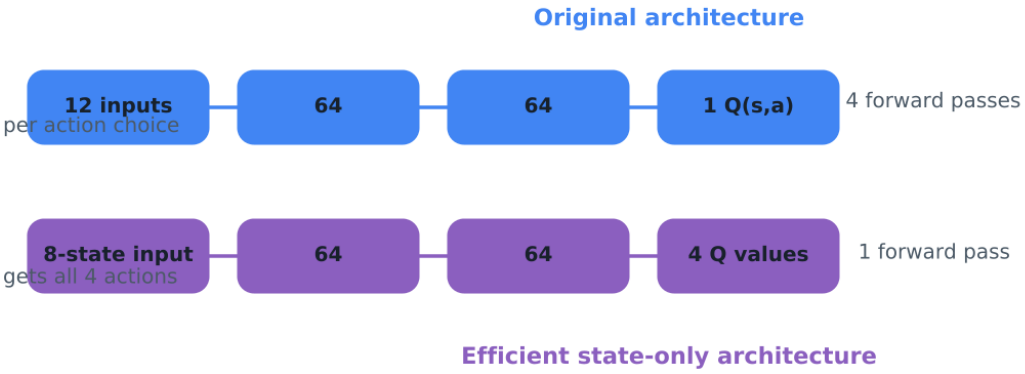


Figure 2. Architecture comparison described in the guide: the original model takes state plus one-hot action; the efficient model takes state only and emits all four action values at once.

Original 12-input model

- Input: 8 state values + 4-dimensional one-hot action representation
- Two fully connected hidden layers with 64 units each
- One scalar output representing Q(s,a)

Bellman target as supervised-learning ground truth

$$Q(s,a) = R(s) + {}^{\text{gamma}} \max Q(s_{\text{prime}}, a_{\text{prime}})$$

The training input is X = [S, A], while the target is Y = R(S) + gamma max Q(S prime, A prime). Experience is collected as tuples (S, A, R(S), S prime).

Where does Q(s prime, a prime) come from?

The guide answers this Socratic question by bootstrapping. Start with randomly initialized weights; the initial network is simply a first guess. Iteratively fitting Bellman targets causes the estimated Q-values to improve over time.

Replay buffer and the basic DQN loop

1. Initialize the neural network parameters randomly.
2. Interact with the Lunar Lander and gather experience tuples.
3. Store the 10,000 most recent transitions in a replay buffer.
4. Construct a training set from the buffer.
5. Compute each Bellman target using the current network.
6. Train a new network with supervised learning and MSE loss.
7. Update the active Q-network.

8. Repeat.

4. Refinement 1 - Efficient Neural Network Architecture

The bottleneck

With four discrete actions, the original 12-input model must perform four forward passes to compare all actions. This is unnecessary compute.

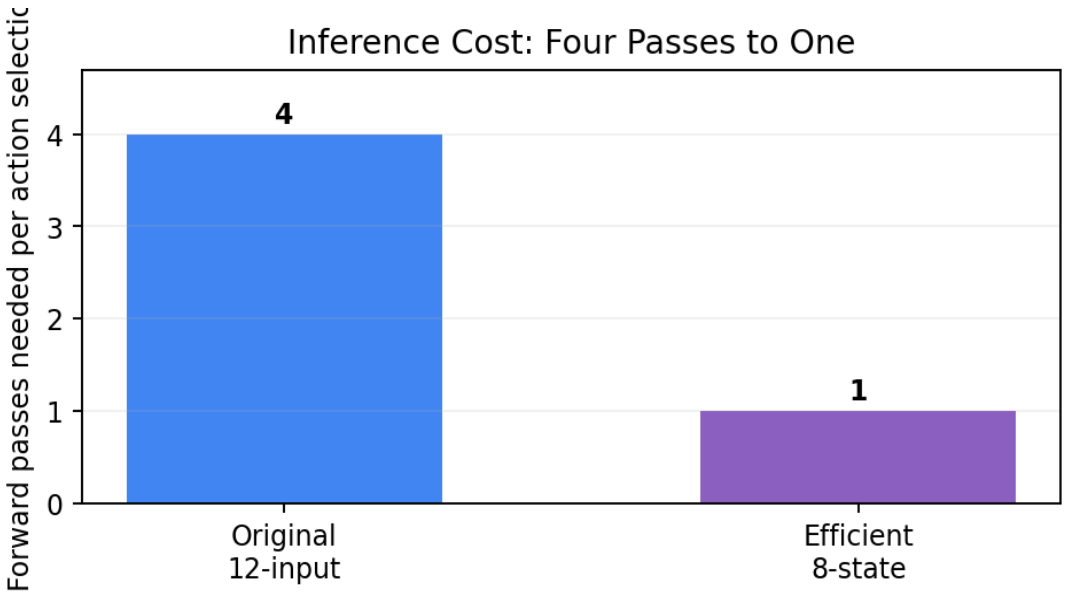


Figure 3. The guide emphasizes a four-pass to one-pass inference improvement.

The solution

- Input: only the 8-dimensional state S
- Two hidden layers of 64 units each
- Four outputs: Q(s, nothing), Q(s, left), Q(s, main), Q(s, right)
- One forward pass produces all action values

This same single pass also makes the Bellman max term efficient because all next-state action values are available together.

5. Refinement 2 - Epsilon-Greedy Exploration

Why greedy-only fails

If the agent always selects the action with the highest current Q estimate, random initialization can create permanent blind spots. A low initial value for the main thruster could cause the agent to avoid it and never discover its importance.

Exploration vs. exploitation

Choice	Probability	Behavior
Exploitation	1 - epsilon	Pick the action with the highest current Q estimate
Exploration	epsilon	Pick randomly among the four actions

High-to-low epsilon schedule

The guide recommends starting with epsilon near 1.0, then decaying it toward a small floor such as 0.01 so the agent transitions from broad exploration to mostly greedy behavior.

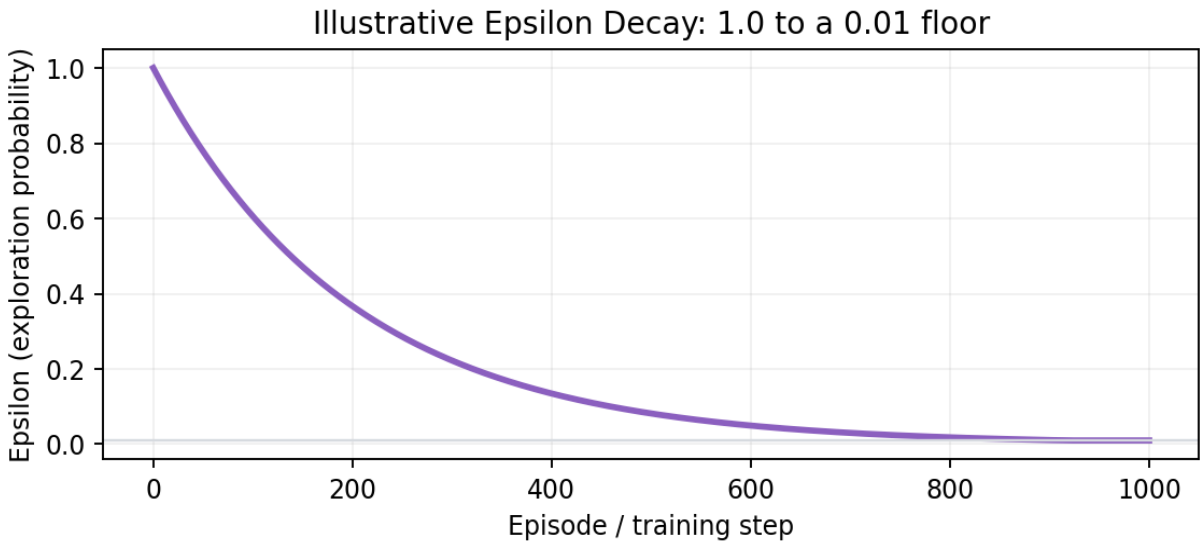


*Figure 4. Illustrative exponential decay corresponding to the study guide's concrete next-step example: epsilon <- max(0.01, epsilon * 0.995).*

```python
epsilon = 1.0
for episode in range(1000):
    epsilon = max(0.01, epsilon * 0.995)
```

Important terminology

The guide notes that "epsilon-greedy" can sound like greedy behavior happens with probability epsilon, but in the described policy the greedy choice happens with probability 1 - epsilon.

Finickiness warning: RL is described as far more sensitive to hyperparameters than supervised learning. A poor epsilon schedule can delay learning dramatically or prevent convergence.

6. Refinements 3 & 4 - Mini-Batches and Soft Updates

Mini-batches

The guide carries the mini-batch idea over from supervised learning: instead of processing all 10,000 replay entries at once, sample a random mini-batch such as 1,000 examples. This adds some noise but greatly reduces the compute per update.

Hard vs. soft updates

A hard update replaces the active parameters abruptly: Q = Q_new. In high-variance RL training, an unlucky mini-batch can therefore create oscillations or divergence.

$$W = 0.01\ W_{new} + 0.99\ W$$
$$B = 0.01\ B_{new} + 0.99\ B$$

The coefficients sum to 1 and act as hyperparameters controlling how gradually the active network adopts newly learned weights.

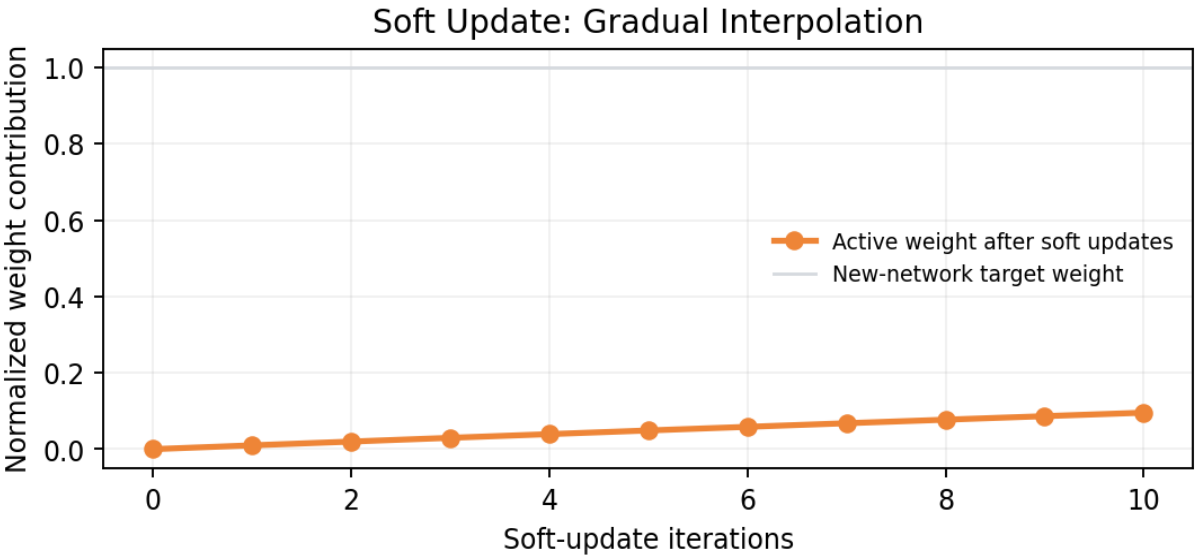


Figure 5. Soft updates gradually move the active parameters toward the newly trained network instead of replacing them all at once.

7. Practical Perspective - Hype vs. Reality

Simulation is easier than the real world

The guide highlights a large sim-to-real gap: many RL results are demonstrated in simulations or video games, while moving the same approach onto a physical robot is substantially harder.

Industrial utility

- There are fewer real-world commercial deployments of RL than of supervised or unsupervised learning.
- For everyday applied machine learning, supervised and unsupervised methods remain the right tools for most jobs.
- RL remains a major pillar of ML with strong future potential, but the guide recommends a pragmatic rather than hype-driven view.

8. Interview Prep & Real-World Applications

1. Continuous state space representation

Interview question: How do you represent a continuous state space, and why can't tabular Q-learning handle it?

Core answer: represent the state as a vector of real-valued numbers. A continuous space has an effectively uncountable/infinite number of possible states, so a table of every state-action pair is not practical. Function approximation, such as a neural network, is used instead.

Application: autonomous flight control can use continuous sensor-derived state variables such as position, orientation, and velocities.

2. Epsilon-greedy trade-off

Interview question: What happens if epsilon = 0 throughout training?

Core answer: the agent never explores. Random initialization can lock in poor preferences for some actions, so essential actions may never be discovered.

Application: recommendation systems can balance exploiting known preferences with exploring new categories.

3. Experience replay

Interview question: Why sample from a replay buffer instead of training in chronological order?

Core answer: sequential transitions are correlated. Random replay sampling breaks those temporal correlations, produces more varied mini-batches, and supports more stable gradient updates.

Application: warehouse robots can reuse varied grasping experiences rather than learning only from the most recent trajectory.

4. Soft updates

Interview question: How does a soft update reduce instability?

Core answer: it integrates new weights gradually, so a noisy update cannot abruptly redefine the policy. The effect is analogous to smoothing policy changes.

Application: the guide uses industrial HVAC optimization as an example where gradual policy changes can help avoid erratic control.

9. Tailored Study Roadmap

Where this lecture fits

The lecture is positioned as a bridge from tabular RL and Bellman equations to modern deep reinforcement learning. Experience replay, epsilon decay, and network stability are described as foundations for later ideas such as Double DQN, Dueling DQN, and continuous-action methods such as DDPG.

Foundational vs. supplementary

Priority	Topic	Why it matters
Must-know	State-only network: S -> 4 outputs	Core DQN architecture used for efficient action selection
Must-know	Epsilon-greedy decay	Controls the shift from exploration to exploitation
Must-know	Bellman target y = R + gamma max Q	Connects RL bootstrapping to supervised learning
Supplementary	Exact helicopter dimensions	Useful robotics context, but domain-specific
Supplementary	Historical DQN context	Helpful background, but not required to implement the method

Common pitfalls

- Epsilon-greedy naming: greedy happens with probability 1 - epsilon, not epsilon.
- Hard vs. soft updates: immediate replacement is not automatically better; smoothing is used for stability.
- Hyperparameter frustration: failure to learn may come from epsilon decay, gamma, or learning-rate choices rather than code logic alone.

Concrete next steps

1. Derive the 12-input and 8-input tensor dimensions by hand.
2. Implement epsilon decay over 1,000 episodes and plot it.
3. Complete the Lunar Lander DQN lab, paying close attention to replay sampling and soft updates.

```python
for episode in range(1000):
    epsilon = max(0.01, epsilon * 0.995)
    # plot epsilon to visualize exploration -> exploitation
```

Takeaway: The central move in this guide is to replace an impossible table over continuous states with a neural network that approximates Q-values, then stabilize learning with replay, exploration scheduling, mini-batches, and gradual weight updates.

One-minute recall

Concept	Remember this
Continuous state	Represent the state with real-valued numbers rather than a finite lookup table.
Lunar Lander	8-dimensional state; 4 discrete actions; reward design encodes landing behavior.
DQN	Use a neural network to approximate Q(s,a) and learn from Bellman targets.
Replay buffer	Keep recent experience and sample random mini-batches instead of training chronologically.
Epsilon-greedy	Explore with probability epsilon; exploit with probability 1 - epsilon.
Soft update	Blend new and old weights so updates are gradual rather than abrupt.

Source basis: supplied study guide. This report preserves its terminology and framing while adding original diagrams and illustrative visualizations for readability.